

AI Ticket Triage Agent

Project Documentation

A browser-based AI agent that reads a customer support ticket, categorizes it, sets its priority, decides whether it needs immediate escalation, assigns a team, and explains its reasoning with a confidence score - flagging anything it cannot confidently classify for human review instead of guessing.

https://github.com/shuklahimanshu20/Saas_ticketing_triage

1. What Problem This Agent Solves

SaaS support teams receive a constant stream of tickets that must be triaged quickly. A small subset of them - live outages, security incidents, data breaches, legal issues - need to bypass the normal queue and get immediate attention. Manually reading and routing every incoming ticket is slow and error-prone. This Agent automates the first, highest-leverage decision in that pipeline: read the ticket, understand it, and decide what should happen to it next.

2. Why This Is an Agent

The system perceives an input (the raw ticket text), reasons about it against a fixed policy (allowed categories, priorities, and a hard-coded critical escalation rule), and autonomously decides an outcome (category, priority, escalation, assigned team, confidence) without a human in the loop for the initial triage step. It also knows the boundary of its own competence: when it is not confident, it deliberately abstains from guessing and routes the ticket to Human Review Required rather than fabricating an answer. That combination - autonomous decision-making plus a built-in refusal path - is what makes this an agent rather than a plain if/else script.

3. How the Agent Works

- The Streamlit UI collects the ticket text and passes it to `analyze_ticket()`.
- If `ANTHROPIC_API_KEY` is configured, the Agent sends the ticket to the Anthropic Claude API with a strict prompt asking for a JSON verdict constrained to the allowed categories, priorities, and escalation states.
- The LLM's JSON response is validated (correct keys, allowed enum values, confidence in range) before it is ever shown to the user. If validation fails, or the API call errors for any reason, the Agent automatically falls back to a deterministic DEMO/MOCK classifier instead of crashing or showing garbage.
- If no API key is configured at all, the Agent runs entirely in DEMO/MOCK mode from the start, and the UI clearly labels this so no one mistakes mock output for a real model call.
- A hard-coded safety rule always forces priority = Critical and escalation = Immediate for Service Outage / Legal-Security categories, even if the LLM's own answer was inconsistent - the critical escalation rule is never left entirely to model judgment.

4. Architecture

```
Streamlit UI (app.py)
  |
  v
agent/ticket_triage_agent.py  <- all business logic, no Streamlit imports
  |
  +-- ANTHROPIC_API_KEY set? --yes--> Anthropic Claude API --> validate
  |                                     |
  |                                     (on failure)
  |                                     v
  +-- no / failure -----> rule-based mock
                           classifier
```

The UI layer and the agent/LLM logic are fully separate: app.py only renders widgets and calls analyze_ticket(); it contains no classification rules and never touches the API key directly.

5. Project Structure

```
ticket-triage-agent/
|
|-- app.py                      Streamlit browser UI
|-- agent/
|   |-- __init__.py
|   |-- ticket_triage_agent.py  Core agent logic (LLM + mock + validation)
|-- data/
|   |-- sample_tickets.json      The 7 sample tickets used by UI and tests
|-- tests/
|   |-- test_triage.py           pytest suite for the 7 required cases
|-- .env.example                Template for ANTHROPIC_API_KEY
|-- .gitignore
|-- requirements.txt
|-- README.md
```

6. Installation

From inside the ticket-triage-agent folder:

```
pip install -r requirements.txt
```

7. Configuring the API Key

The Agent reads the key from the ANTHROPIC_API_KEY environment variable. It is never hard-coded anywhere in the source.

Option A - .env file (local development)

```
cp .env.example .env
# then edit .env and paste your real key
```

Option B - shell environment variable

```
# Windows PowerShell
$env:ANTHROPIC_API_KEY = "your_api_key_here"

# macOS/Linux
export ANTHROPIC_API_KEY="your_api_key_here"
```

If no key is present, the app automatically and visibly runs in DEMO/MOCK mode using a deterministic rule-based classifier - it is always usable and demonstrable without any API key or cost.

8. Running the Browser Application Locally

```
streamlit run app.py
```

Streamlit prints a local URL (typically <http://localhost:8501>) - open that in your browser.

9. Deploying It Live (Streamlit Community Cloud)

The project is already pushed to GitHub, which is the only prerequisite for Streamlit Community Cloud (a free, official hosting service built for exactly this kind of app). Steps to deploy:

- Go to share.streamlit.io and sign in with your GitHub account.
- Click "Create app" -> "Deploy a public app from GitHub".
- Repository: shuklahimanshu20/Saas_ticketing_triage
- Branch: main
- Main file path: app.py
- Optional: under "Advanced settings" -> "Secrets", add ANTHROPIC_API_KEY = "your_real_key_here" to enable LIVE mode. Skipping this leaves the deployed app in DEMO/MOCK mode, which is still a valid, fully working demonstration.
- Click "Deploy". After a short build, Streamlit gives a permanent public URL (e.g. <https://saas-ticketing-triage-xxxx.streamlit.app>) that works without your machine running.

GitHub repository: https://github.com/shuklahimanshu20/Saas_ticketing_triage

10. Example Tickets and Expected Outputs

Sample	Ticket text	Category	Priority	Escalation
Login problem	I forgot my password and cannot login.	Account/Login	Normal	Standard
Billing problem	I was charged incorrectly for my subscription.	Billing	Normal	Standard
Feature request	Can you please add dark mode?	Feature Request	Normal	Standard
Bug	The application crashes whenever I upload a PDF.	Bug	Normal/High	Standard
Widespread outage	Nobody in our organization can access the platform. The service is down for everyone.	Service Outage	Critical	IMMEDIATE
Security/data breach	We believe our account has been hacked and customer data may have been exposed.	Legal/Security	Critical	IMMEDIATE
Unclear ticket	Hello, I need help.	Unknown	Normal	HUMAN REVIEW REQUIRED

All seven are pre-loaded via the "Load Sample Ticket" buttons in the UI and also covered by the automated test suite.

11. Escalation Logic

If the ticket text indicates any of: a live service outage, widespread system failure, a legal issue, a security issue, a data breach, or hacking, the Agent always sets priority = Critical and escalation = Immediate, and the UI shows a prominent red "IMMEDIATE ESCALATION" banner along with the reason. This rule is enforced in code even when using the live LLM, so a borderline or inconsistent model answer cannot silently skip it.

12. Human Review Logic

If the Agent cannot confidently classify a ticket - the text is too vague, empty, or does not match any recognizable pattern - it sets category = Unknown and escalation = Human Review Required with a low confidence score, and the UI shows a clear amber warning banner. The Agent never fabricates a category or reason it is not confident about.

13. Running the Tests

pytest

The tests force DEMO/MOCK mode (by clearing ANTHROPIC_API_KEY) so they are deterministic, free, and do not require network access. They cover all 7 required test cases from the assignment spec, and all 7 pass.

14. AI Tools Used to Build This Project

This project was built with Claude Code (Anthropic), using the Claude Sonnet 5 model, based on a detailed specification provided by the developer. Claude Code was used to scaffold the project structure, write the agent logic, the Streamlit UI, the test suite, the README, and this documentation PDF.

15. What Was Manually Reviewed or Modified

- The keyword rule-set in the mock classifier was reviewed against all 7 required test cases to confirm each one resolves to the expected category, priority, and escalation.
- The critical escalation rule was deliberately hard-coded as a non-overridable safety check, rather than trusted entirely to LLM output, so it holds even in live mode.
- The app was updated to also read the API key from Streamlit Cloud's Secrets manager (st.secrets) so it works identically whether run locally or deployed.
- The .env.example and DEMO/MOCK UI messaging were checked against the actual analyze_ticket() fallback behavior to ensure the mode banner always accurately reflects what is happening under the hood.

16. Scope Note

This is a single, independent triage agent built for demonstration purposes. It does not connect to any real billing, engineering, or support system - all team assignment is illustrative, and sample data is mocked. It does not perform end-to-end workflow orchestration.